detected, it brings the cluster back to the desired state by applying the necessary changes.

Benefits of GitOps

Embracing GitOps empowers modern teams to achieve greater efficiency and reliability in their software delivery processes.

Simplified operations: GitOps simplifies the deployment and management of applications by using familiar Git workflows, reducing the need for complex manual operations.

Consistency and reproducibility: By managing configurations as code in Git, GitOps ensures that the same configuration used in development is applied to staging and production environments, leading to consistency and reproducibility.

Version control and collaboration: Git provides version control, enabling teams to collaborate effectively, review changes, and maintain a history of system modifications.

Automated auditing and compliance: GitOps enables easier auditing and compliance tracking since all changes are recorded in Git repositories.

GitOps tools

GitOps tools provide continuous delivery and deployment and adhere to the GitOps approach, where the desired state of the infrastructure and applications is version-controlled in Git repositories. These tools continuously monitor the repositories for changes and automatically apply those changes to the target environment, typically a Kubernetes cluster. Though several open source tools like Jenkins-x, Spinnaker, Kustomize and GitLab are available, we will quickly look into the workings of two widely adopted tools — Argo CD and Flux.

Argo CD

Argo CD is an open source continuous delivery tool that provides a powerful and efficient way to manage Kubernetes applications, offering automated continuous delivery and the benefits of the GitOps workflow that runs on top of Kubernetes.

With Argo CD, you can define your application configurations declaratively in Git repositories and have Argo CD automatically reconcile the state of your applications to match the desired state defined in those repositories. It is widely adopted by organisations looking to simplify application deployments, reduce manual interventions, and improve the overall reliability of their Kubernetes environments.

How Argo CD works

Application definition: You define your application specifications, including the deployment manifests, service configurations, and any other required resources, in a Git repository.

Argo CD server: The Argo CD server continuously monitors the Git repository for changes. When a change is detected, the server initiates a reconciliation process.

Reconciliation: During reconciliation, Argo CD compares the desired state in the Git repository with the actual state in the Kubernetes cluster. If there are differences, Argo CD makes the necessary changes to bring the cluster in line with the desired state.

Application deployment: Argo CD deploys and manages your applications using Kubernetes manifests, Helm charts, or other supported application definition formats.

Application synchronisation: Argo CD keeps the applications continuously synchronised with the Git repository. Any changes made directly to the cluster are automatically reverted, ensuring that the desired state remains consistent.

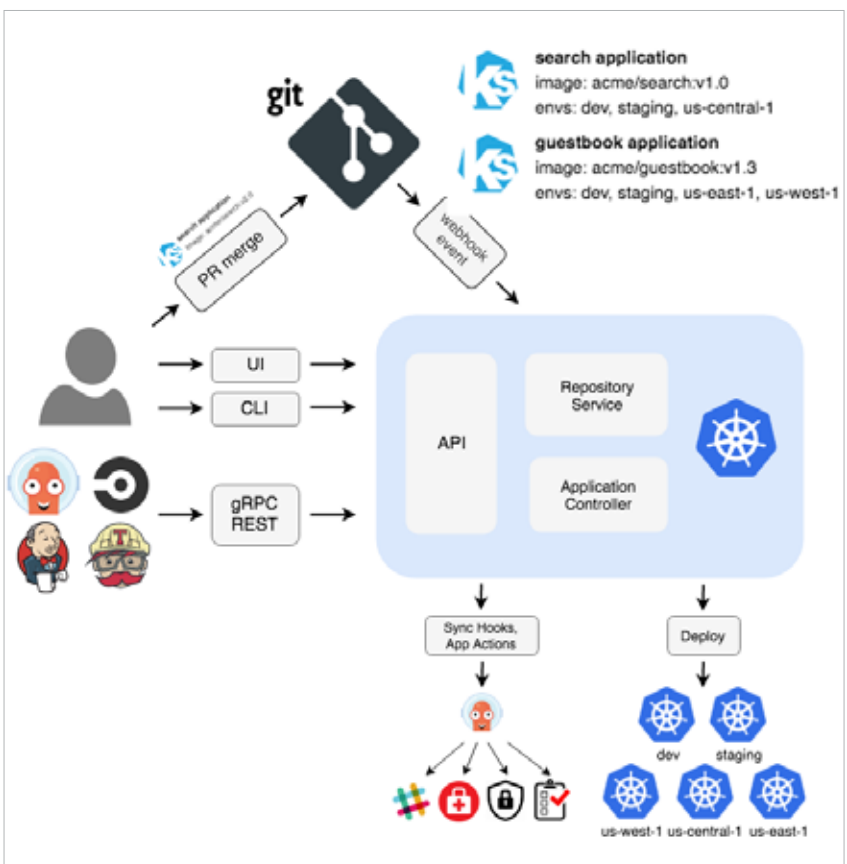

Figure 2: ArgoCD